Building Resilient Distributed Systems

PDC Assignment 2 — StudySync Architecture Analysis & Redesign

Name: Faizan Amir | **Student ID:** BSCS23212 | **Date:** May 10, 2026

1. Root Cause Analysis

1.1 Problem 1 — Lost Update (Synchronization)

The root cause is the complete absence of concurrency control at both the database and API layers. When User A and User B both issue a `GET /document/:id`, they each receive the same snapshot of the resource at version V_n. Both independently mutate their local copies and then issue a `PUT` or `PATCH`. FastAPI processes these as two independent, uncoordinated transactions: whichever write arrives second silently overwrites the first the classic **Lost Update anomaly**.

Because there is no `version` column, no row-level lock, and no compare-and-swap in the `UPDATE` statement, the database has no mechanism to detect that the row was concurrently modified between the read and the write. The overwrite happens without error and without notification, leaving one user's changes permanently destroyed.

1.2 Problem 2 — Dropped Webhook (Coordination)

The webhook handler is **stateless and fire-and-forget**. Clerk delivers the `subscription.cancelled` event exactly once over HTTP. If the request is dropped by a network blip, or the server is briefly unavailable (a restart, a traffic spike), the event is silently discarded. Clerk retries a limited number of times within a short window; after that, the event is gone.

Because the handler stores no **idempotency key** and maintains no persistent event log, there is no way to replay or audit missed events. The outcome is permanent state divergence: Clerk records the user as free-tier while the application database still holds `is_premium = TRUE`.

1.3 Problem 3 — Blocking LLM Call (Fault Tolerance)

The route handler calls the external LLM API using a synchronous `requests.get()`, which **blocks the executing thread** for the full duration of the request up to 60 seconds on timeout. There is no timeout cap, no circuit breaker, and no fallback. A single slow LLM response occupies a Uvicorn worker for the entire wait. Under any meaningful load, all workers fill up with hanging requests and the application becomes unresponsive for every user. This is a **cascading failure**: an upstream dependency's outage propagates into a complete application outage.

2. Architectural Redesign

2.1 Fix 1 — Optimistic Locking with Versioned Writes

Add an integer `version` column to the `documents` table. Every read returns the current version alongside the payload. Every write must supply the version it read, and the `UPDATE` is conditioned on:

```
UPDATE documents SET content = :c, version = version + 1
    WHERE id = :id AND version = :expected_version
```

If the row was modified concurrently, the `WHERE` clause matches zero rows. The application raises **HTTP 409 Conflict** and the client re-fetches before retrying. No pessimistic locks are held, so throughput remains high for the common non-conflicting case.

Schema change: `ALTER TABLE documents ADD COLUMN version INTEGER NOT NULL DEFAULT 1;`

UML Sequence Diagram — Two Concurrent Users

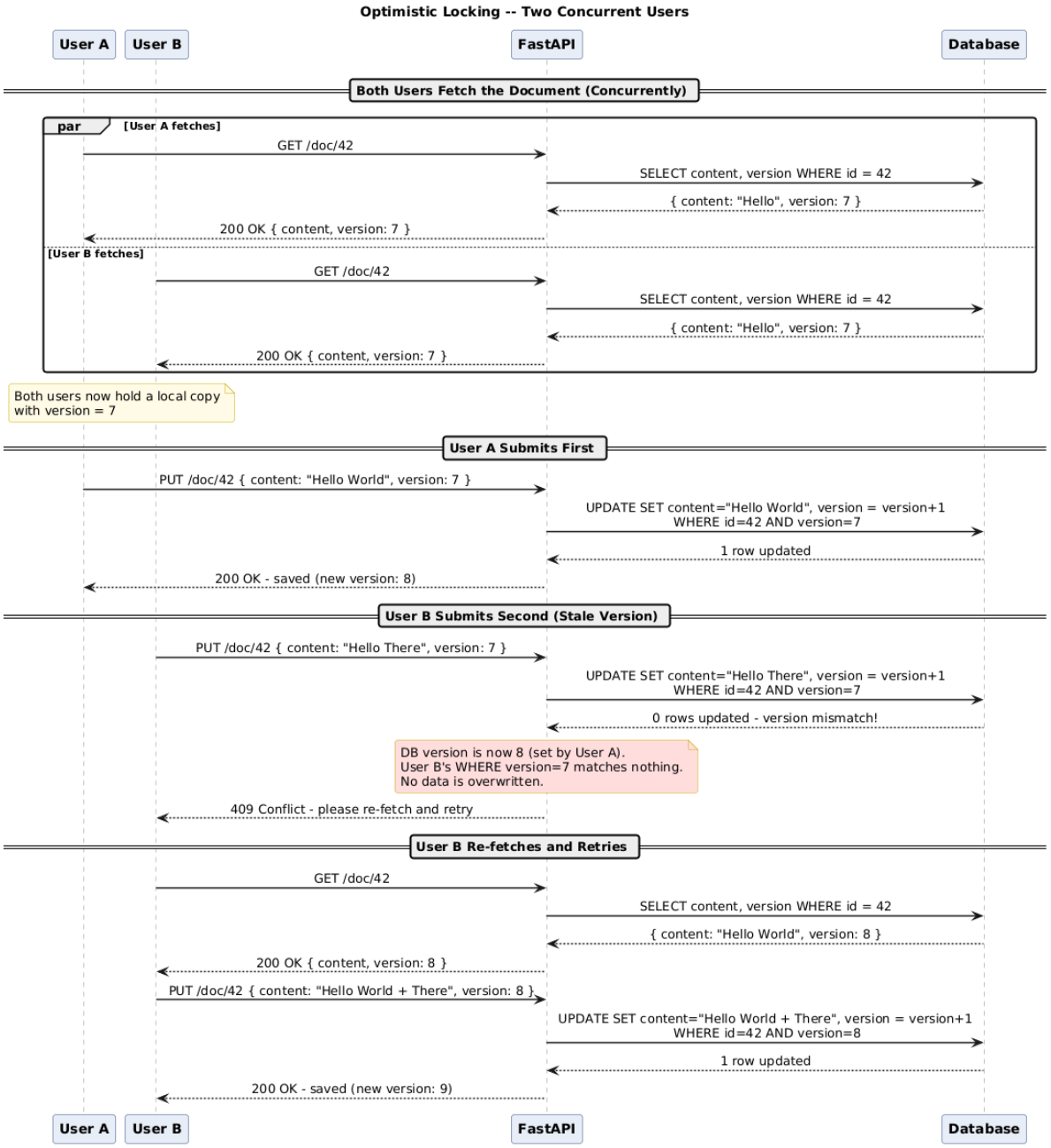

Figure 1: Optimistic locking: User A wins; User B receives 409 Conflict and must re-fetch.

2.2 Fix 2 — Idempotent Webhook Handler with Dead-Letter Queue

Two mechanisms together eliminate dropped-event risk:

- **Idempotency key store.** On every incoming webhook, extract Clerk's unique event ID (`evt.id`) and attempt an `INSERT` into a `processed_events` table with a `UNIQUE` constraint on the event ID. If the `INSERT` succeeds, process the event; if it raises a duplicate-key error, return `200 OK` immediately the event was already handled. This makes the handler fully safe to replay.
- **Persistent retry queue with DLQ.** Wrap the handler in a `try/except`. On any processing failure, push the raw event payload to a message queue (Redis Streams, RabbitMQ, or AWS SQS). A background consumer retries with exponential back-off. After N failures the event moves to a **Dead-Letter Queue (DLQ)** for manual inspection, guaranteeing no cancellation is ever silently lost.

2.3 Fix 3 — Circuit Breaker with Fallback

Implement a three-state Circuit Breaker around every LLM call:

State	Behaviour
CLOSED	Normal operation. Failures are counted toward the threshold.
OPEN	Threshold exceeded. Requests fail-fast; fallback is returned immediately. No calls reach the LLM.
HALF-OPEN	After a recovery timeout, one probe request is allowed. Success → CLOSED; failure → OPEN.

All LLM calls must also use an explicit hard timeout (`httpx.AsyncClient(timeout=5.0)`) so no single request can ever block for 60 seconds regardless of circuit state. The fallback returns a cached or user-friendly response, keeping the application available even during full LLM outages.

2.4 CAP Theorem Trade-offs

Fix	Consistency	Availability	Rationale
Optimistic Locking	Strong	Slightly reduced	Conflict retries add latency but guarantee no lost updates. A pure AP system (e.g. CRDTs) would trade consistency for availability, which is unacceptable for shared documents.
Webhook DLQ	Eventual	High	Events are guaranteed to be processed eventually (CP-leaning for critical subscription state) but not instantaneously. Acceptable since billing does not require millisecond consistency.
Circuit Breaker	Reduced (stale fallback)	High	Deliberately favours AP: the system stays available with a degraded response rather than blocking. Correct trade-off for a non-critical AI feature , users prefer a fast partial response over a frozen UI.

In summary: optimistic locking leans **CP**, the circuit breaker leans **AP**, and the webhook DLQ provides **eventual consistency** , together a balanced strategy that applies the right consistency model to each subsystem based on its criticality.